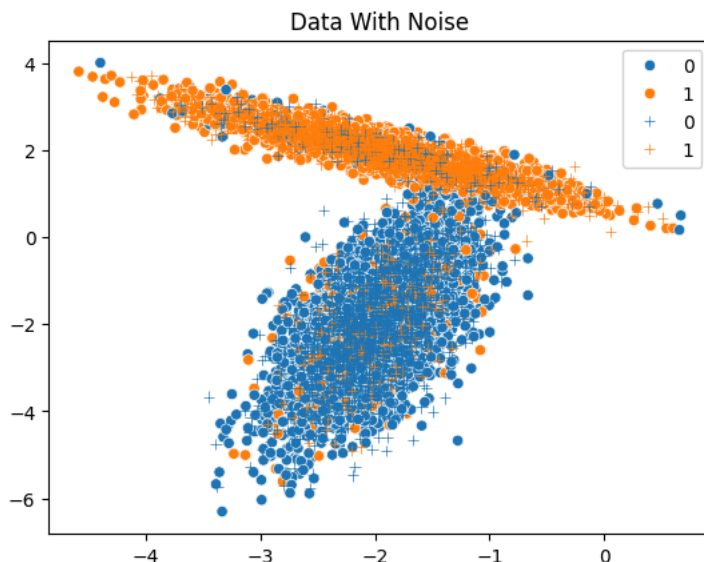


```

from sklearn.datasets import make_classification # для генерации данных
from sklearn.model_selection import train_test_split # для разделения на обучение и тест
from sklearn.ensemble import RandomForestClassifier # случайный лес
import numpy as np
import seaborn as sns # для простого отображения
import matplotlib.pyplot as plt #
# создаем данные
X,y = make_classification(n_samples=10000, # число примеров
n_features=2, # число признаков (атрибутов)
n_informative=2, # из них информативных
n_redundant=0, # из них не информативных
n_repeated=0, # из них повторяющихся
n_classes=2, # число классов
n_clusters_per_class=1, # число кластеров на класс
class_sep=2, # влияет на расстояние между кластерами
flip_y=0.2, # 0.2 доля ошибок (шума)
weights=[0.5,0.5], # пропорции числа данных в классах
random_state=17) #
# разделяем на обучающие и тестовые, случайно
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.33, random_state=42)
# рисуем данные
plt.subplots();#
ax1=plt.gca();#
sns.scatterplot(x=X_train[:,0],y=X_train[:,1],hue=y_train,ax=ax1);# обучающие
sns.scatterplot(x=X_test[:,0],y=X_test[:,1],hue=y_test,ax=ax1,marker="+");# тестовые
ax1.set_title("Data With Noise");#
plt.show();#

```



```

# Создаем классификатор на основе случайного леса. Изменяйте параметры и смотрите как это влияет на обу
clf = RandomForestClassifier(max_depth=5,# 5 максимальная глубина дерева
n_estimators=5,# 10 число деревьев в лесу
max_features=1)# максимальное число признаков для каждого дерева
#from sklearn.tree import DecisionTreeClassifier
#clf=DecisionTreeClassifier(max_depth=5)
clf.fit(X_train, y_train) # обучаем
y_pred = clf.predict(X_test) # проверяем на тестовых данных
score=clf.score(X_test, y_test) # считаем среднюю точность
print(score)
ind=y_test==y_pred; # индексы совпадений результата классификации и меток классов
plt.subplots();
ax2=plt.gca();
# рисуем "правильно" распознанные примеры
sns.scatterplot(x=X_test[ind,0],y=X_test[ind,1],hue=y_test[ind],ax=ax2);
# рисуем "неправильно" распознанные примеры
sns.scatterplot(x=X_test[~ind,0],y=X_test[~ind,1],hue=y_pred[~ind],ax=ax2,marker="+");
#sns.scatterplot(x=X_test[:,0],y=X_test[:,1],hue=y_test[:,],ax=ax2,marker="+");
ax2.set_title("With Noise");
# считаем и рисуем разделяющую поверхность.
plot_step=0.01
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1 # немного измененные минимальное и максимальные знач
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1 # немного измененные минимальное и максимальные знач
# считаем прямоугольную сетку возможных значений этих атрибутов
xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step), #

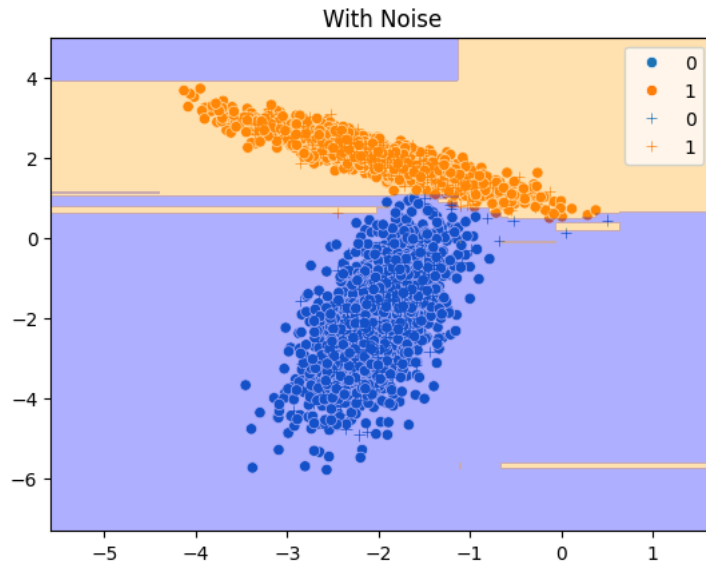
```

```

np.arange(y_min, y_max, plot_step)) #
# считаем выход классификатора для всех примеров сетки
# не забыв что массивы данных нужно привести к требуемому размеру.
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape) # и преобразуем обратно в исходному размеру
# рисуем разделяющую поверхность
0.8945454545454545
cs = plt.contourf(xx, yy, Z, levels=1, colors=['blue','orange'],alpha=0.3) # рисуем контурную карту
plt.show();
#clf.estimators_[0].n_features_in_

```

0.8942424242424243



```

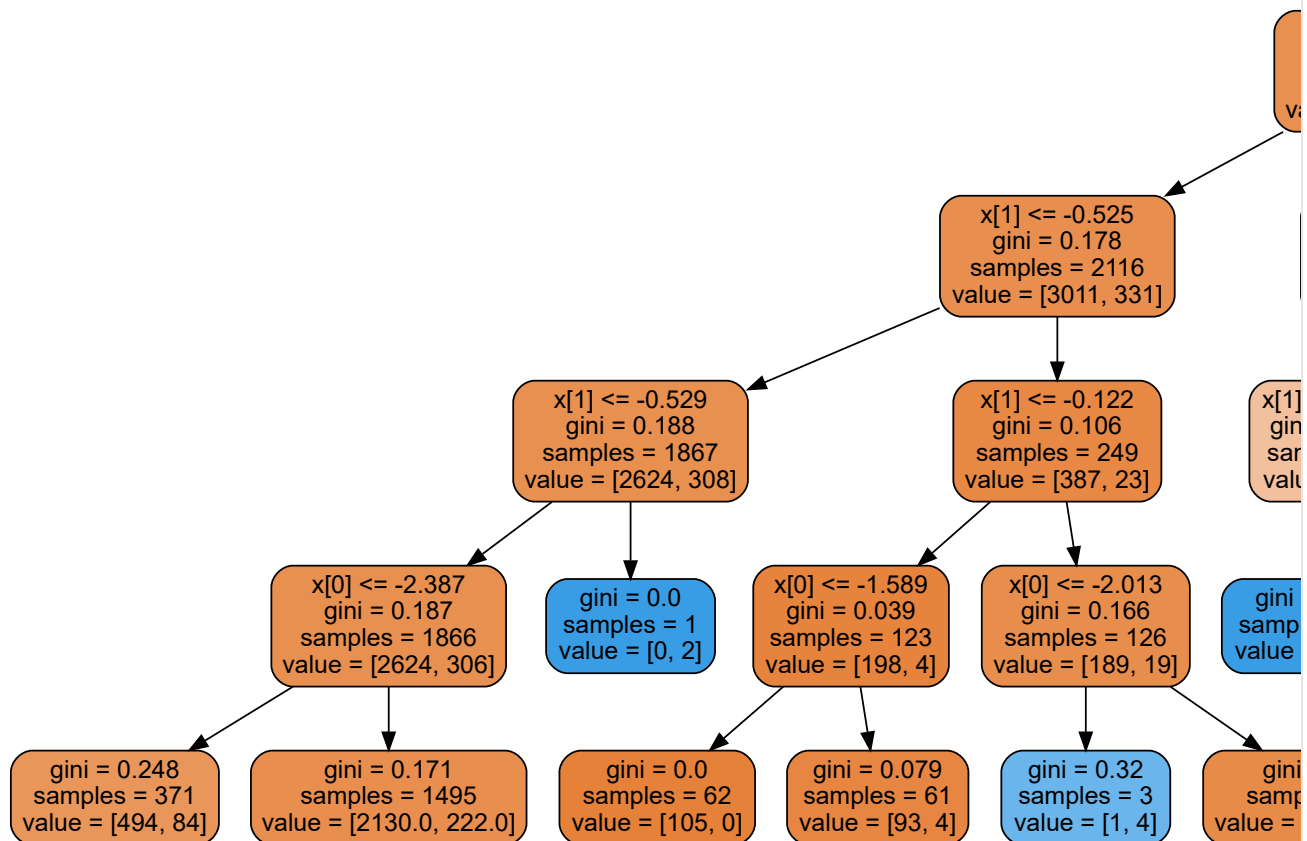
tree_data=clf.estimators_[2] # третье дерево
# рисуем его
from sklearn import tree
import graphviz
dot_data = tree.export_graphviz(tree_data, out_file=None, # можем указать дополнительные опции конверт
filled=True, rounded=True) # прочие детали отображения
graph = graphviz.Source(dot_data) # # загружаем дерево из переменной или файла в представление graphviz

```

```

tree_data=clf.estimators_[4] # третье дерево
# рисуем его
from sklearn import tree
import graphviz
dot_data = tree.export_graphviz(tree_data, out_file=None, # можем указать дополнительные опции конверта
filled=True, rounded=True) # прочие детали отображения
graph = graphviz.Source(dot_data) # # загружаем дерево из переменной или файла в представление graphviz
graph # отображаем на экране

```



```

from sklearn.datasets import load_wine
# загружаем данные
data_wine = load_wine()
# атрибуты
print("Features: ", data_wine.feature_names)
# метки классов
print("Labels: ", data_wine.target_names)
Accuracy= 0.9814814814814815
F1= 0.9817689085981769
Precision= 0.9803921568627452
Recall= 0.9841269841269842
#
X = data_wine.data
y = data_wine.target
# разделяем на обучающие и тестовые
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
# создаем классификатор
clf = RandomForestClassifier(max_depth=5, n_estimators=10, max_features=2)
# обучаем его
clf.fit(X_train, y_train)
# проверяем на тестовых данных
pred = clf.predict(X_test)
# считаем метрики
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
def plot_confusion_matrix(clf,x,y):
    predictions = clf.predict(x)
    cm = confusion_matrix(y, predictions, labels=clf.classes_)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    display_labels=clf.classes_)
    disp.plot()
    plt.show()
accuracy = accuracy_score(y_test, pred)
f1 = f1_score(y_test, pred, average="macro")
precision = precision_score(y_test, pred, average="macro")
recall=recall_score(y_test, pred, average="macro")
print('Accuracy= ', accuracy)

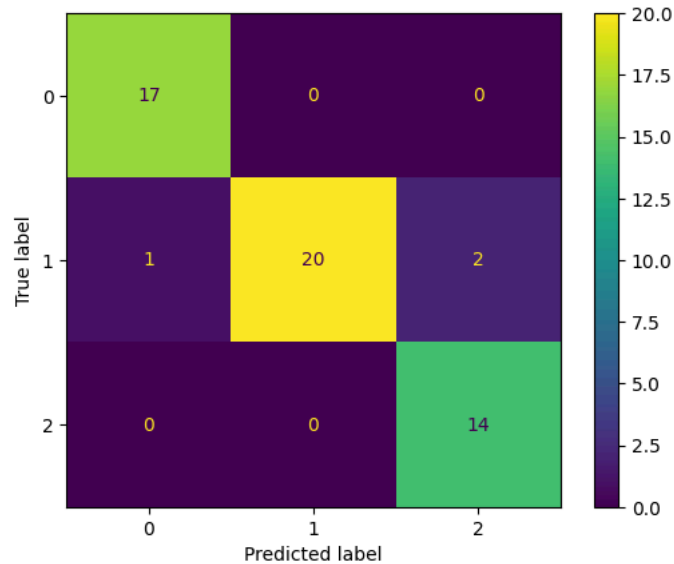
```

```

print('Accuracy=', accuracy,
      print('F1=', f1)
print('Precision=', precision)
print('Recall=', recall)
plot_confusion_matrix(clf, X_test, y_test)

```

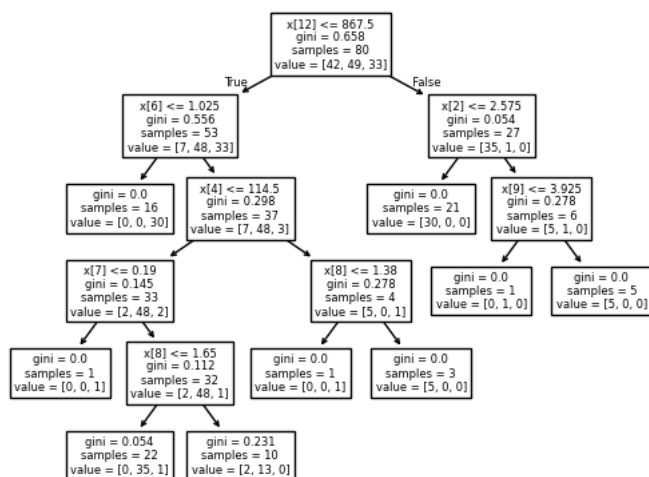
Features: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_pher']
 Labels: ['class_0' 'class_1' 'class_2']
 Accuracy= 0.9444444444444444
 F1= 0.94499815430048
 Precision= 0.9398148148148149
 Recall= 0.9565217391304347



```

# выбираем дерево из леса
estimator = clf.estimators_[2]
# рисуем его
import graphviz
from sklearn import tree
tree.plot_tree(estimator);

```



```

tree_data=tree.export_graphviz(estimator, out_file=None,
feature_names = data_wine.feature_names,
class_names = data_wine.target_names,
rounded = True, proportion = False,
precision = 2, filled = True);
graph=graphviz.Source(tree_data,format='png');
graph

```

